



CHAINWAY LABS

**Citrea: Method ID Security Council & Base
Fee Update
Security Assessment Report**

Version: 2.0

October, 2025

Contents

Introduction	2
Disclaimer	2
Document Structure	2
Overview	2
Security Assessment Summary	3
Scope	3
Approach	3
Coverage Limitations	3
Findings Summary	3
Detailed Findings	5
Summary of Findings	6
Mempool Base Fee Configuration Uses Latest Fork Instead Of Current Fork	7
Cross-Chain Replay Attack On BatchProofMethodId Updates	9
Miscellaneous General Comments	11
A Vulnerability Severity Classification	13

Introduction

Sigma Prime was commercially engaged to perform a time-boxed security review of the Chainway Labs components in scope. The review focused solely on the security aspects of these components, though general recommendations and informational comments are also provided.

Disclaimer

Sigma Prime makes all effort but holds no responsibility for the findings of this security review. Sigma Prime does not provide any guarantees relating to the function of the components in scope. Sigma Prime makes no judgements on, or provides any security review, regarding the underlying business model or the individuals involved in the project.

Document Structure

The first section provides an overview of the functionality of the Chainway Labs components contained within the scope of the security review. A summary followed by a detailed review of the discovered vulnerabilities is then given which assigns each vulnerability a severity rating (see [Vulnerability Severity Classification](#)), an *open/closed/resolved* status and a recommendation. Additionally, findings which do not have direct security implications (but are potentially of interest) are marked as *informational*.

The appendix provides additional documentation, including the severity matrix used to classify vulnerabilities within the Chainway Labs components in scope.

Overview

Citrea is the first rollup that enhances Bitcoin blockspace using zero-knowledge technology, providing a fully EVM-compatible layer that uses Bitcoin for both data availability and settlement through a trust-minimized two-way peg program called Clementine. The platform enables Ethereum-compatible smart contracts to execute on Bitcoin's infrastructure while maintaining security and decentralization.

This security review focused on two primary code changes. The first introduces a security council mechanism for batch proof method ID upgrades, requiring 3-of-5 multisignature validation from designated council members. The second reduces the minimum base fee from 0.01 gwei to 0.001 gwei in the `Fork3` upgrade, lowering transaction costs for users while maintaining network economic security.

Security Assessment Summary

Scope

The review was conducted on the files hosted on the [chainwayxyz/citrea](#) repository.

The scope of this time-boxed review was strictly limited to changes in PR [#2878](#) at commit [e907e59](#).

The fixes of the identified issues were assessed at commit [e7a3b8a](#).

Note: third party libraries and dependencies were excluded from the scope of this assessment.

Approach

The security assessment covered components written in Rust.

The manual review focused on identifying issues associated with the business logic implementation of the components in scope. This includes their internal interactions, intended functionality and correct implementation with respect to the underlying functionality of the Rust language.

Additionally, the manual review process focused on identifying vulnerabilities related to known Rust anti-patterns and attack vectors, such as unsafe code blocks, integer overflow, floating point underflow, deadlocking, error handling, memory and CPU exhaustion attacks, and various panic scenarios including index out of bounds, `panic!()`, `unwrap()`, and `unreachable!()` calls.

To support the Rust components of the review, the testing team may use the following automated testing tools:

- Clippy linting: <https://doc.rust-lang.org/stable/clippy/index.html>
- Cargo Audit: <https://github.com/RustSec/rustsec/tree/main/cargo-audit>
- Cargo Outdated: <https://github.com/kbknapp/cargo-outdated>
- Cargo Geiger: <https://github.com/rust-secure-code/cargo-geiger>
- Cargo Tarpaulin: <https://crates.io/crates/cargo-tarpaulin>

Output for these automated tools is available upon request.

Coverage Limitations

Due to the time-boxed nature of this review, all documented vulnerabilities reflect best effort within the allotted, limited engagement time. As such, Sigma Prime recommends to further investigate areas of the code, and any related functionality, where majority of critical and high risk vulnerabilities were identified.

Findings Summary

The testing team identified a total of 3 issues during this assessment. Categorised by their severity:

- Low: 1 issue.
- Informational: 2 issues.

Detailed Findings

This section provides a detailed description of the vulnerabilities identified within the Chainway Labs components in scope. Each vulnerability has a severity classification which is determined from the likelihood and impact of each issue by the matrix given in the Appendix: [Vulnerability Severity Classification](#).

A number of additional properties of the components, including optimisations, are also described in this section and are labelled as “informational”.

Each vulnerability is also assigned a **status**:

- **Open**: the issue has not been addressed by the project team.
- **Resolved**: the issue was acknowledged by the project team and updates to the affected components(s) have been made to mitigate the related risk.
- **Closed**: the issue was acknowledged by the project team but no further actions have been taken.

Summary of Findings

ID	Description	Severity	Status
CMF3-01	Mempool Base Fee Configuration Uses Latest Fork Instead Of Current Fork	Low	Closed
CMF3-02	Cross-Chain Replay Attack On BatchProofMethodId Updates	Informational	Resolved
CMF3-03	Miscellaneous General Comments	Informational	Resolved

CMF3-01	Mempool Base Fee Configuration Uses Latest Fork Instead Of Current Fork		
Asset	crates/sequencer/src/mempool.rs		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Medium

Description

The mempool configuration hardcodes the minimum protocol base fee to the latest fork specification instead of using the current active fork.

The sequencer's mempool is configured with a minimum protocol base fee using `SpecId::latest()` :

```
crates/sequencer/src/mempool.rs::CitreaMempool::new()

let pool_config = PoolConfig {
    pending_limit: SubPoolLimit {
        max_txs: mempool_conf.pending_tx_limit as usize,
        max_size: (mempool_conf.pending_tx_size * 1024 * 1024) as usize,
    },
    basefee_limit: SubPoolLimit {
        max_txs: mempool_conf.base_fee_tx_limit as usize,
        max_size: (mempool_conf.base_fee_tx_size * 1024 * 1024) as usize,
    },
    queued_limit: SubPoolLimit {
        max_txs: mempool_conf.queue_tx_limit as usize,
        max_size: (mempool_conf.queue_tx_size * 1024 * 1024) as usize,
    },
    blob_limit: SubPoolLimit {
        max_txs: 0,
        max_size: 0,
    },
    max_account_slots: mempool_conf.max_account_slots as usize,
    minimal_protocol_basefee: min_base_fee_per_gas(SpecId::latest()),
    ..Default::default()
};
```

The `SpecId::latest()` function always returns `Fork3`, regardless of the current active fork.

```
crates/sovereign-sdk/rollup-interface/src/spec.rs::SpecId::latest()

impl SpecId {
    /// Get the latest active (official) SpecId.
    pub const fn latest() -> Self {
        Self::Fork3
    }
}
```

The `min_base_fee_per_gas()` function returns different minimum base fees depending on the fork:

```
crates/primitives/src/constants.rs::min_base_fee_per_gas()

pub fn min_base_fee_per_gas(spec_id: SpecId) -> u64 {
    if spec_id >= SpecId::Fork3 {
        1_000_000 // 0.001 gwei
    } else {
        10_000_000 // 0.01 gwei
    }
}
```


This means that on networks running earlier fork specifications (e.g., Genesis, Kumquat, or Tangerine), the mempool will incorrectly allow transactions with base fees as low as 0.001 gwei instead of the current fork's minimum of 0.01 gwei. This could lead to the mempool accepting transactions that are technically invalid for the current network fork.

The impact is low as this primarily affects transaction admission to the mempool rather than consensus validity. However, it represents a configuration mismatch that could cause unexpected behaviour in pre-Fork3 networks. The likelihood is medium as it affects any deployment running fork specifications earlier than Fork3.

Recommendations

Replace `SpecId::latest()` with the current active fork specification when configuring the mempool. The sequencer should obtain the current fork specification from the fork manager and use that to determine the appropriate minimum base fee for mempool configuration.

Resolution

The development team has acknowledged the issue with the following comment:

"This issue only causes transactions to wait between sequencer update and fork activation. The gas fee-related endpoints would still return the correct values, hence a user must deliberately send with a lower base fee to have their transaction stuck."

CMF3-02	Cross-Chain Replay Attack On BatchProofMethodId Updates	
Asset	crates/sovereign-sdk/rollup-interface/src/state_machine/da.rs, crates/light-client-prover/src/circuit/method_id_verifier.rs	
Status	Resolved: See Resolution	
Rating	Informational	

Description

Security council signatures for `BatchProofMethodId` updates can be replayed across different networks, enabling an attacker to activate unauthorised method IDs on production networks using signatures intended for test environments.

The `BatchProofMethodIdBody` structure contains only the method ID and activation height but critically omits any network identifier:

```
crates/sovereign-sdk/rollup-interface/src/state_machine/da.rs::BatchProofMethodIdBody
```

```
#[derive(Debug, Clone, Serialize, Deserialize, Eq, PartialEq, BorshDeserialize, BorshSerialize)]
pub struct BatchProofMethodIdBody {
    /// New method id of upcoming fork
    pub method_id: [u32; 8],
    /// Activation L2 height of the new method id
    pub activation_l2_height: u64,
}
```

When security council members sign a `BatchProofMethodId` update, they sign the serialised `BatchProofMethodIdBody` with EIP-191 prefixing. The signature verification process in `verify_method_id_security_council()` validates three of five council member signatures but never binds these signatures to a specific network:

```
crates/light-client-prover/src/circuit/method_id_verifier.rs::verify_method_id_security_council()
```

```
pub fn verify_method_id_security_council(
    initial_da_pubkeys: [[u8; 33]; 5],
    msg: &[u8],
    signatures_with_idx: &[[u8; 64], u8; 3],
) -> bool {
    // EIP-191 prefix + keccak256 -> 32-byte prehash
    // @audit No chain ID or network identifier included in signature
    let prehash = eip191_hash_message(msg);

    // ... signature verification logic ...
}
```

The vulnerability exists when different Citrea networks use identical or overlapping security council public keys. For example, if Testnet and Mainnet use the same security council public keys, an attacker could capture these signatures from Testnet's DA layer and replay them on Mainnet. Since Mainnet would use the same security council public keys and the signatures contain no network-specific information, the signatures would verify successfully, allowing the attacker to activate the potentially untested or vulnerable method ID on Mainnet at the same activation height.

This issue has an informational severity rating as it was discovered by the development team during the testing phase.

The issue itself would have a high impact rating as successful replay attacks could activate unauthorised or malicious batch proof method IDs on production networks, potentially compromising the integrity of the entire rollup's state transition verification. This could enable invalid state transitions to be accepted or cause network-wide disruption if an incompatible method ID is activated.

The likelihood would be medium as the attack requires that different networks share the same security council public keys and that security council members sign method ID updates on one network that could be used maliciously on another. It requires specific timing and the presence of valid signatures on a source network.

Recommendations

Add a `chain_id` field to `BatchProofMethodIdBody` that must match the L2 chain ID configured in the EVM chain configuration. This ensures that signatures are cryptographically bound to a specific network and cannot be replayed across chains even if they share security council members.

Ensure that the signature verification logic in `verify_method_id_security_council()` validates that the signed message includes the correct chain ID for the current chain.

Resolution

The development team has added a `chain_id` field to `BatchProofMethodIdBody` that must match the chain ID of the current chain.

This issue has been resolved in PR [#2915](#) and [#2878](#).

CMF3-03	Miscellaneous General Comments
Asset	crates/*
Status	Resolved: See Resolution
Rating	Informational

Description

This section details miscellaneous findings discovered by the testing team that do not have direct security implications:

1. Inefficient Duplicate Index Checking Implementation

Related Asset(s): *crates/light-client-prover/src/circuit/method_id_verifier.rs*

The duplicate signature index validation in `verify_method_id_security_council()` uses hardcoded pairwise comparisons that are inefficient and inflexible for threshold changes. The current implementation explicitly checks all combinations of three indices:

```
crates/light-client-prover/src/circuit/method_id_verifier.rs::verify_method_id_security_council()

// Check for duplicate indices
if signatures_with_idx[0].1 == signatures_with_idx[1].1
    || signatures_with_idx[0].1 == signatures_with_idx[2].1
    || signatures_with_idx[1].1 == signatures_with_idx[2].1
{
    log!("Duplicate signature indexes found");
    return false;
}
```

This approach is hardcoded for exactly three signatures and would require modification if the threshold changes from 3-of-5 to another value like 4-of-5. A more efficient and flexible approach would be to verify that the indices are provided in ascending order, which inherently prevents duplicates whilst supporting any threshold value.

2. Outdated Comments And Documentation

Related Asset(s): *crates/light-client-prover/src/circuit/method_id_verifier.rs, crates/bitcoin-da/src/verifier.rs*

Multiple locations contain outdated comments that no longer reflect the current implementation, creating potential confusion for developers.

In `verify_method_id_security_council()`, the function documentation states that signatures must be in the same order as the initial values constants:

```
crates/light-client-prover/src/circuit/method_id_verifier.rs

/// The signatures should be in the same order as the one in the initial values constants.
```

However, since the introduction of the `(signature, index)` tuple structure, signatures no longer need to be in the same order as the constants, making this documentation misleading.

Additionally, in the Bitcoin DA verifier, a comment references an outdated struct name:

```
crates/bitcoin-da/src/verifier.rs

// The signature verification of BatchProverMethodId is done in the circuit
ParsedTransaction::BatchProofMethodId(method_id) => {
```

The comment refers to `BatchProverMethodId` but the actual struct is now called `BatchProofMethodId`.

Update all outdated comments and documentation to accurately reflect the current implementation. This includes revising the function documentation for `verify_method_id_security_council()` to remove references to ordering requirements and correcting struct name references in comments.

3. Magic Numbers Used In Security Council Configuration

Related Asset(s): `crates/light-client-prover/src/circuit/method_id_verifier.rs`,
`crates/light-client-prover/src/circuit/initial_values.rs`,
`crates/sovereign-sdk/rollup-interface/src/state_machine/da.rs`

The security council verification implementation uses hardcoded magic numbers throughout the codebase instead of named constants, reducing code maintainability and readability.

The most prominent examples occur in the `verify_method_id_security_council()` function signature, where `5` represents the total number of security council public keys and `3` represents the signature threshold:

```
crates/light-client-prover/src/circuit/method_id_verifier.rs::verify_method_id_security_council()
```

```
pub fn verify_method_id_security_council(
    initial_da_pubkeys: [[u8; 33]; 5],
    msg: [u8],
    signatures_with_idx: [[u8; 64], u8]; 3],
) -> bool {
```

Similarly, the `BatchProofMethodId` struct definition contains hardcoded values:

```
crates/sovereign-sdk/rollup-interface/src/state_machine/da.rs::BatchProofMethodId
```

```
pub struct BatchProofMethodId {
    pub body: BatchProofMethodIdBody,
    pub signatures_with_index: [[u8; 64], u8]; 3],
}
```

Additional magic numbers found throughout the codebase include `33` (bytes per compressed public key), `64` (bytes per signature), and bounds checking with hardcoded values like `if index >= 5`. These magic numbers appear in function parameters, constant definitions across multiple network configurations, helper functions, and test code.

Using magic numbers instead of named constants creates maintenance challenges if the security council configuration changes, such as transitioning from a 3-of-5 to 4-of-7 threshold, which would require manually updating numerous hardcoded values across multiple files.

Consider replacing all magic numbers with named constants such as `SECURITY_COUNCIL_MEMBERS`, `SIGNATURE_THRESHOLD`, `COMPRESSED_PUBKEY_SIZE`, and `SIGNATURE_SIZE` to improve code readability and maintainability.

Recommendations

Ensure that the comments are understood and acknowledged, and consider implementing the suggestions above.

Resolution

The development team have acknowledged these findings, addressing them where appropriate as follows:

1. **Inefficient Duplicate Index Checking Implementation:** Implemented as recommended in commit [24200bd](#).
2. **Outdated Comments And Documentation:** Implemented as recommended in commits [5e15b03](#) and [459910e](#).
3. **Magic Numbers Used In Security Council Configuration:** Implemented as recommended in commit [4d4fc71](#).

Appendix A Vulnerability Severity Classification

This security review classifies vulnerabilities based on their potential impact and likelihood of occurrence. The total severity of a vulnerability is derived from these two metrics based on the following matrix.

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Table 1: Severity Matrix - How the severity of a vulnerability is given based on the *impact* and the *likelihood* of a vulnerability.

σ'